

ForeFlight Hackathon 2025



ForeFlight

Introduction

This hackathon is inspired by one of the challenges faced by the Runway Analysis team here at ForeFlight. The problem is called *Obstacle Filtering*, and at its core is about choosing the most important obstacles that must be considered when deciding whether an airplane can safely take off from a runway.

Essentially, the problem is that we often have a lot of obstacle data for a given runway, but the modules that compute the take-off performance of the airplane take a very limited number of obstacles as input (usually 20-30). This means that we need to filter the obstacles to select the ones that are most critical to the flight path. An example of this can be seen in *Figure 1*. It contains more than 100 obstacles, but this has been narrowed down to the selected obstacles seen in dark blue.

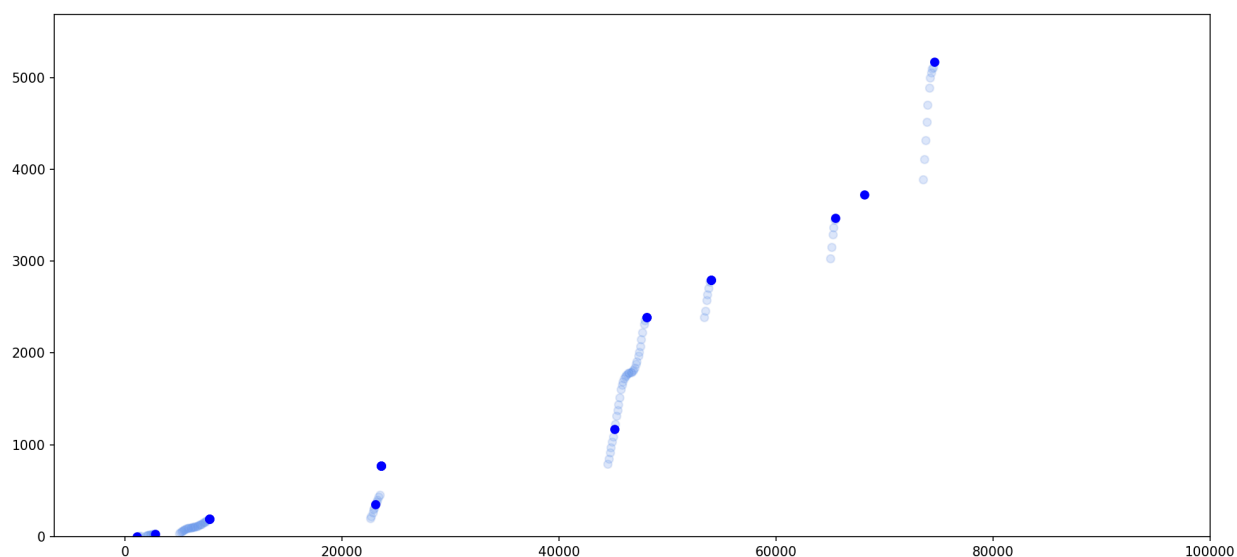


Figure 1: A plot showing a side view of obstacles in front of a runway, with distance on the x-axis and height on the y-axis. The filtered obstacles are shown in dark blue.

We have to be careful when filtering the obstacles, since we want to ensure that the performance calculations of the airplane are still safe and correct even after we have removed obstacles. That is, if a flight path was impossible before the data was filtered, it must also be impossible after the filtering. We say that a filtering is *conservative* if this holds for every possible flight path of the airplane.

An example to demonstrate this can be seen in *Figure 2*. Here we see a single flight path represented by the green line. This flight path is not possible to execute with the true data, since it collides with the first obstacle. This simply means that the plane cannot take off in this instance. It also means that a filtering which removed the first obstacle in this case would lead to a non-conservative filtering, since it would suddenly allow the flight path.

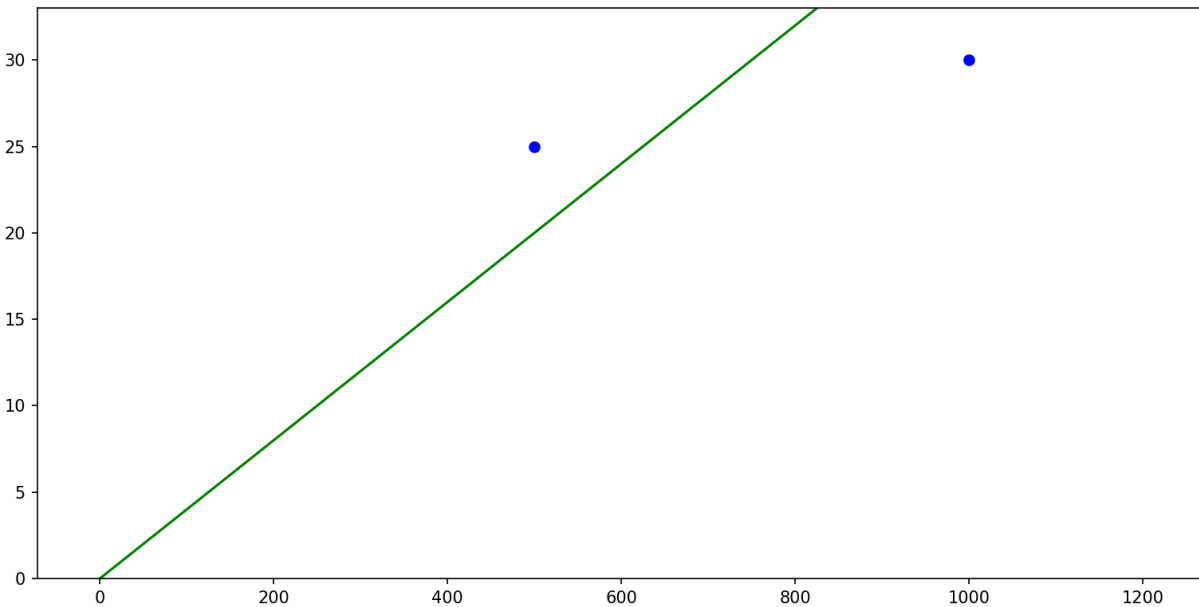


Figure 2: An example of a flight path colliding with an obstacle.

Challenges

The challenges increase in difficulty. You are free to skip over a challenge if you find another more interesting, however some of the challenges build on each other and might be easier to do in order.

We recommend reading the entire document before diving into the challenges.

We will walk around and talk to each group often, and if you are stuck, it's better to ask for help than to not make any progress at all.

Get Hacky

This is a hackathon, and here you are allowed to hack some ugly code together! If some aspect of a challenge seems difficult to solve in an exact way, feel free to find a method that gets a close-enough result. This is often how real software is built.

You are also free to use whatever programming language and frameworks you want. We recommend you to use a language that you are comfortable with and where it isn't too difficult to visualize your data.

Judging

When deciding a winner, we will look at how far you have gotten, how you have solved the challenges, etc. As some of these challenges are open-ended, being creative and making cool stuff gives extra points!

While there is a prize to be won, the most important for us is that you have fun with the challenges. Some of them may be difficult to solve, so don't stress if you don't manage to solve all of them (the challenges are designed so that you likely won't run out of things to do).

We recommend making a new script for each challenge such that you can go back and demonstrate/explain your solutions for each of the challenges.

Challenge 1 - Visualization

You are given a csv file containing the relevant obstacle data for a runway. The file is called GWJI.csv and can be found in the folder “challenge_1_2”. The file contains a header with two columns. Each row after the header contains data about an obstacle. The ‘elevation’ column is the relative elevation to the runway in feet, whereas ‘distance’ is the distance from the end of the runway in feet. The obstacles are arranged in ascending order of distance.

Your challenge is to load the data from the file and visualize it by plotting it.

We recommend that you plot regularly throughout the remaining challenges.

Challenge 2 - Step Filter

A simple obstacle filter is the so-called *step filter*, where the idea is to remove any point that is lower than or at the same height as a previous point. The assumption we make here is that the plane is always either climbing or remaining at the same level. Since it never descends, any obstacles lower than an earlier obstacle is thus irrelevant to consider.

Your challenge is to apply the step filter to obstacles in the same file as the previous challenge.

We strongly recommend that you simply read from the given file and output your results either to another file or as a visualisation. That is, avoid modifying the original file.

We also recommend that you for the remaining challenges apply this filter to the data first.

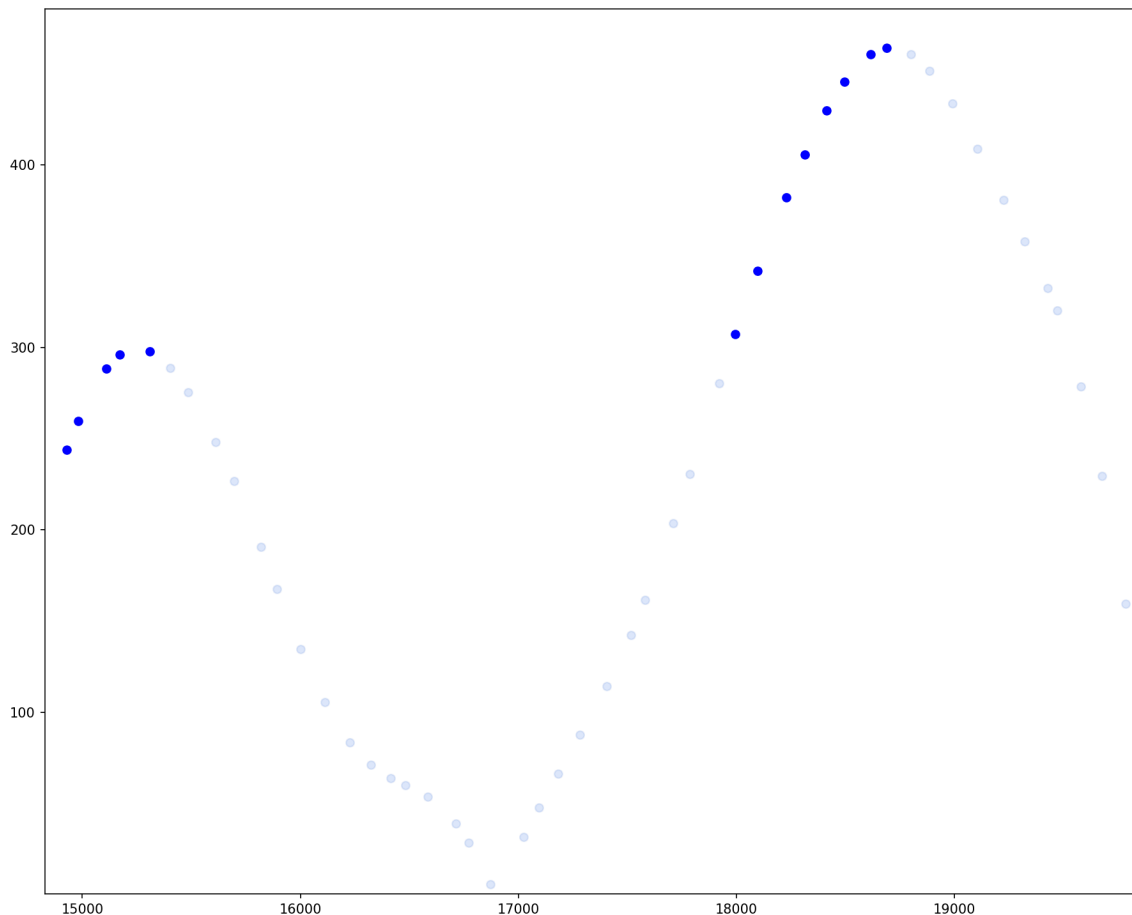


Figure 3: An example of a section of obstacles where the step filter has been applied. Points that are kept are shown in dark blue and the points that have been removed are shown in light blue.

Note:

For the remaining challenges, all assumptions of the flight path of the airplane will be given. The assumptions of a previous challenge do not necessarily apply to the next. These assumptions are not always true in the real world, but serve to provide interesting scenarios for the different challenges.

Challenge 3

For this challenge the assumptions are as follows.

Assumptions:

- The airplane does not descend.
- The airplane takes off at coordinate (0,0).
- The airplane has a constant climb rate (gradient) for the entire climb. This value can differ between airplanes, but it is constant for a given climb.

The 3 relevant obstacle files for this challenge are located in the “challenge_3” folder. Each file represents the obstacles of a runway at an airport.

Your challenge is to create an algorithm for filtering the obstacles in a conservative manner using the given assumptions. The fewer obstacles the better.

Hint: For this challenge an optimal algorithm can select very few points.

Challenge 4

For this challenge, we take into account that an airplane does not always take off from the exact same spot on the runway.

Assumptions:

- The airplane does not descend.
- The airplane takes off somewhere between $(-6000,0)$ and $(0,0)$.
- The airplane has a constant climb rate for the entire climb. This value can differ between airplanes, but it is constant for a given climb.

The relevant files for this challenge are located in the “challenge_4” folder.

Your challenge is to create an algorithm for filtering all the obstacle files in a conservative manner using the given assumptions. The fewer obstacles the better.

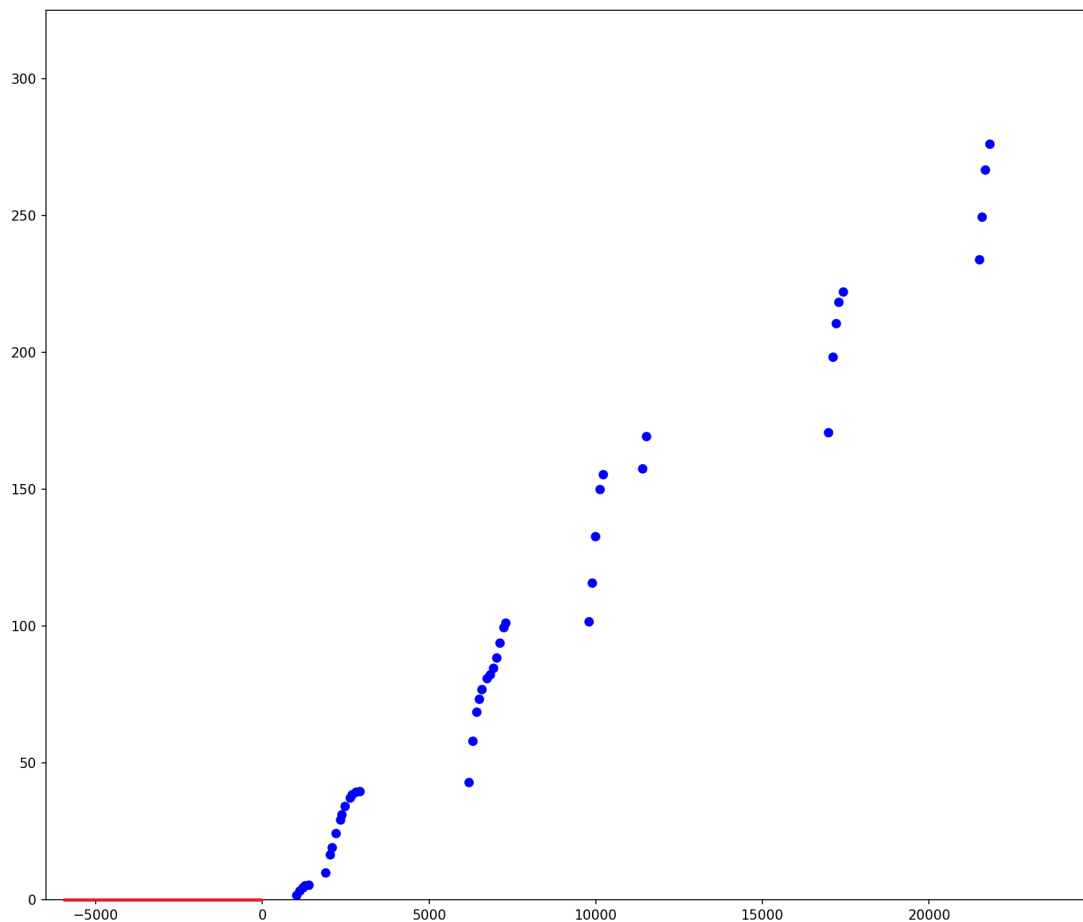


Figure 4: An illustration showing the approximate location of the runway in red. We assume that the airplane can take off from any point along the red line.

Challenge 5

This is the last challenge and is thus a bit more open ended. You are encouraged to try different approaches and iterate over your solution.

The data you are given for this challenge has already been step-filtered. This was done to limit the file sizes and to decrease the time it takes to run the evaluation script.

Assumptions:

- The airplane does not descend.
- The airplane takes off at coordinate (0,0).
- The flight path of the airplane is divided into 3 sections in the given order:
 - The first segment has a climb gradient between 2% and 20%, and has a (x-axis) length of 1000 feet or more.
 - The acceleration segment, where the airplane does not climb at all, but simply continues at the same altitude. This segment has a (x-axis) length of 1000 feet or more.
 - The final segment has a climb gradient between 2% and 20%, and continues until the x-value of the last data point.
- The gradients and lengths of the different segments are independent of each other

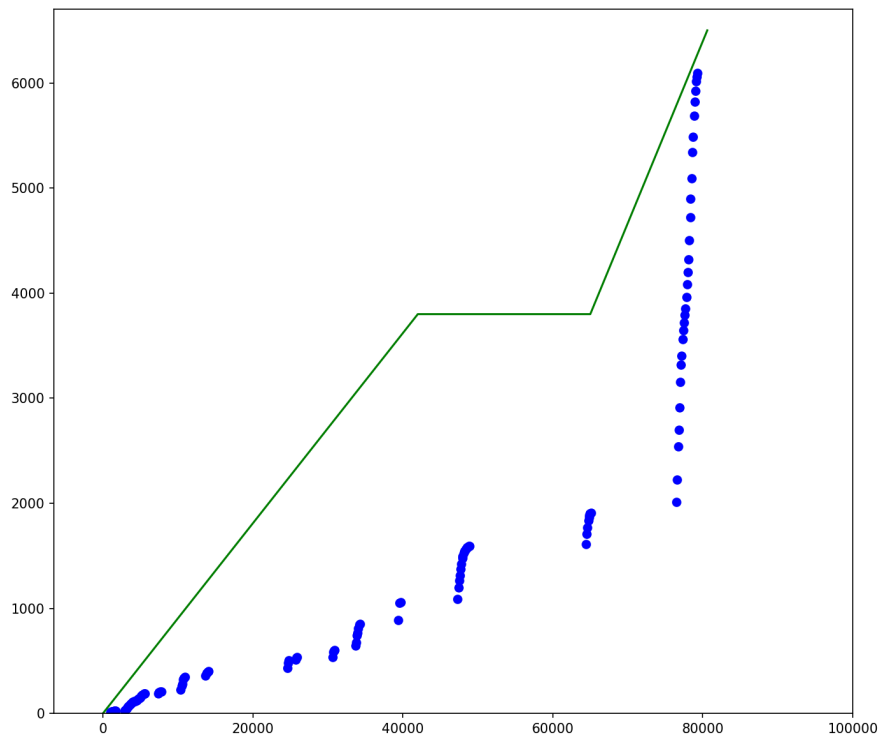


Figure 5: An example of a flight path for this challenge is shown here in green. The first segment is the initial climb, followed by the acceleration segment which is the flat part of the line. The last part is the final segment.

You are allowed to not only remove points, but also create new points in the output. This can be helpful as it allows you to ‘merge’ two points into one, by removing the two points and adding a new point in a place that ‘covers’ where the two points were. Of course this still needs to be safe and conservative to do so.

When this is a possibility it is of course easy to make a conservative filter. However, we also want the filter to be good. Here ‘good’ is a measure of how many of the possible flight paths are restricted by the filter (this will be explained in detail in the following sections).

Your challenge is to create an algorithm for filtering all the obstacle files in a conservative manner using the given assumptions. The algorithm is not allowed to output more than 10 points for each file. The algorithm should try to minimize the number of possible flight paths it restricts.

The relevant obstacle files can be found in the “challenge_5” folder, as well as an evaluation python script.

Evaluation script:

The evaluation script can be run against the filtered obstacles to compare how good it is in relation to the original obstacle file, and thus to evaluate the quality of your filtering. It simply simulates some n randomly generated flight paths. Of these n flight paths some will collide with the terrain and some will not. If the flight path collides with both the original obstacles and filtered obstacles it is ignored.

Otherwise the following interesting cases are considered:

- Collides with filtered obstacles, but not the original obstacles: *Limiting*
- Collides only with original obstacles but not with the filtered obstacles: *Non-conservative*

Essentially, if a flight path leads to a non-conservative case, it means that the obstacle filtering is not safe, since a flight path exists where it would collide with the terrain, but the filtered obstacles would ignore this. It is critical to avoid this case when implementing your filtering algorithm.

If a flight path leads to a limiting case, it means that this flight path would technically be possible to fly in reality, but that the obstacle filter is restrictive and doesn’t allow it. In some sense this is unavoidable, and serves more as a ‘score’ of how efficient the obstacle filtering is, where the fewer limiting cases the better.

The evaluation script outputs percentages for both non-conservative and limiting cases. Using the script is optional and mainly serves as a tool for you to evaluate your solutions. This is also the script we will run to evaluate your solutions.

Running the script:

The evaluation script can be run by running `python evaluate_filter.py` where your filtered obstacles are located in a folder named ‘filtered_obstacles’ in the same directory as the script. It can also be run with different options. E.g.,

```
python evaluate_filter.py --validation_count 10000 --seed 1234 --plot
```

will consider 10000 flight paths for each file, seed the random generator with 1234, and show the resulting plot. The more flight paths you consider, the more precise the result is going to be, but it will also take longer to run. The result is printed and also saved to a generated file `results.csv`.

The script requires `numpy`, `matplotlib` and `pandas` to run.

Since we need to run the script against your solutions, we need access to the filtered files you create for this challenge.

Requirements for output:

For each obstacle file, you should output a csv file with the same filename containing the filtered obstacles. These should be output in a folder called “filtered_obstacles”. The csv files you output should be in the same format as the files given. Once the time is up, you should zip this “filtered” folder containing only the csv files of filtered obstacles, and send the zipped file to the email: troels@foreflight.com

The deadline for this is 15:00.